Diploma in UX Design

Jakob Nielsen's 10 principles of UI

Summary Notes

Contents

3 Lesson objectives

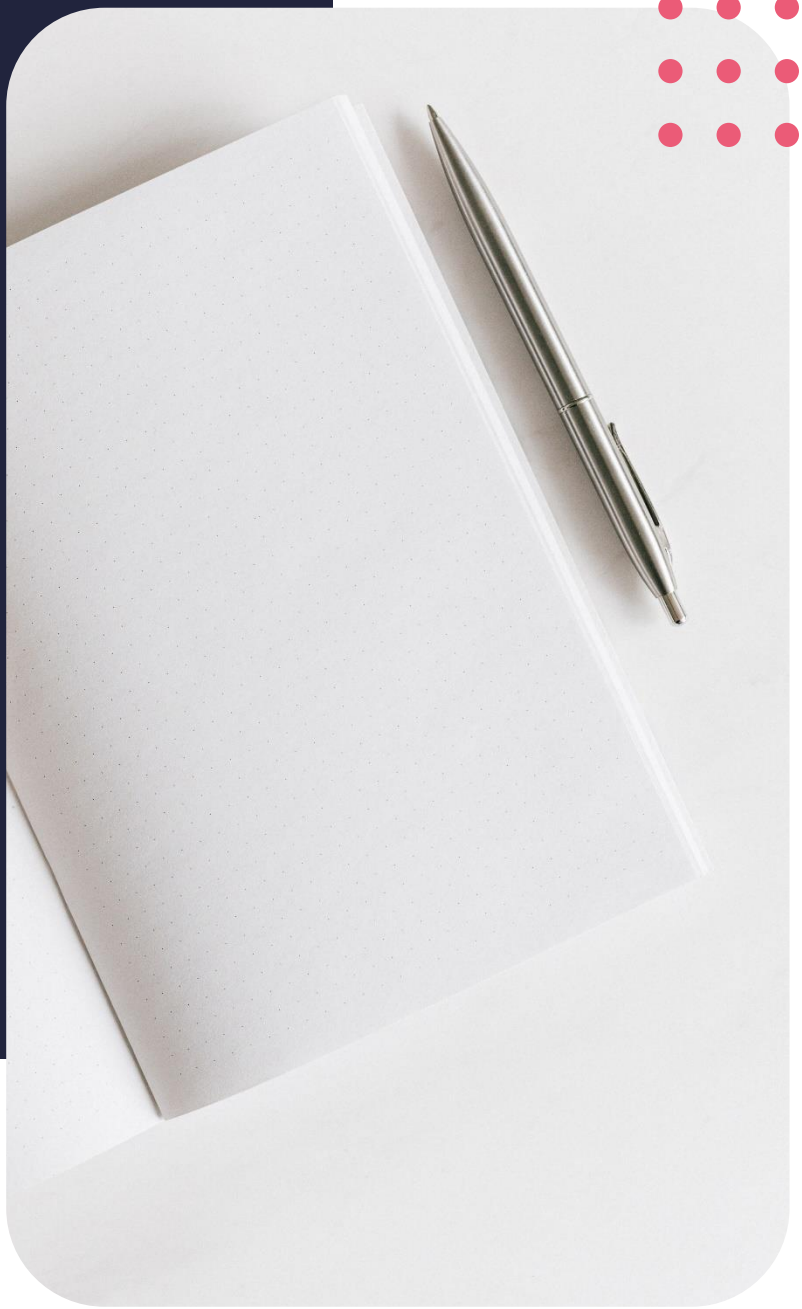
3 Introduction

3 Deep dive into Jakob Nielsen's 10 principles of UI

21 Lesson challenge

21 Conclusion

21 References



Lesson objectives

By the end of this lesson, you should be able to:

- Recap Jakob Nielsen's general principles of UI
- Understand the principles of UI
- List examples of where these principles have been applied

Introduction

In our previous lesson, we've learned that there are several frameworks or checklists of good practices for UX design and that the most common one is Jakob Nielsen's 10 general principles for interaction design. We briefly touched on them but in this lesson, we will delve deeper into what these principles are and why they are important.

Let's briefly recap on what these principles are as they are fundamental to your knowledge of UX design.

- Visibility of system status
- Match between system and the real world
- User control and freedom
- Consistency and standards
- Error prevention
- Recognition rather than recall
- Flexibility and efficiency of use
- Aesthetic and minimalist design
- Helping users to recognise, diagnose, and recover from errors

Kindly note that in future lessons we will investigate other frameworks that are commonly also used to perform UX evaluations or audits.

Deep dive into Jakob Nielsen's 10 principles

1. Visibility of system status

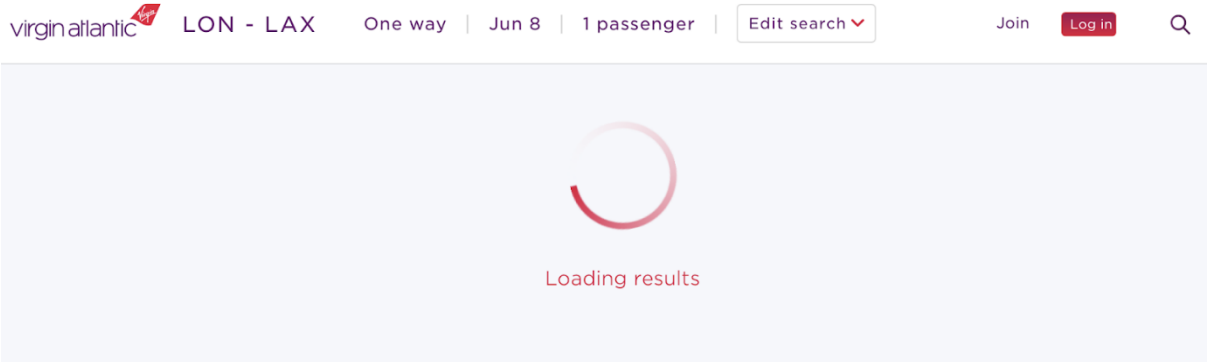
The system must always let the user know what is going on, by providing immediate feedback. Every user action must have an appropriate reaction, very much like a natural conversation between 2 people.

When the speaker says something, the listener must provide feedback confirming that the message was received, either verbally or non-verbally. Good listening skills may include saying something back, nodding one's head, smiling or whatever is appropriate now.

The same applies to human-computer interaction. The user is the speaker, and the app or website is the listener or vice versa. If the user presses a button, something must happen to indicate that the button press was picked up. This can be indicated by a ripple effect or a sound for example. Then a reaction will need to take place for example, going to the next page or a success message that appears.

For longer processes, a progress indicator or spinner must be displayed while the step runs in the background. A multiple steps process requires a step progress indicator which must be displayed indicating how far the user is in the process and how long the remaining steps will take.

Here you can see a typical progress spinner from on the Virgin Atlantic site that is displayed while searching for flights, which will only take a few seconds.

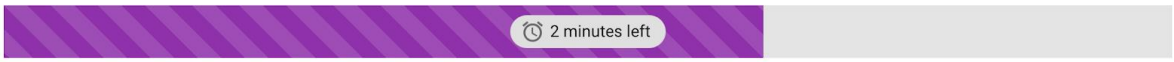


Ideally, feedback should be instant, however, the animation is often handy to make it clear that change or action is taking place.

Often providing immediate results is not possible because the system has to fetch data from a distant database, or complex, time-consuming processing has to happen at the backend. If a process will take less than a second, no animation, or more preferably, a simple fading-in-or-out effect may be most appropriate. If the result may take one to eight seconds, a spinner may be the best. A spinner doesn't need to be plain like the one shown earlier, it can occur in a variety of formats like shown here, often animated to show that something is happening in the background.

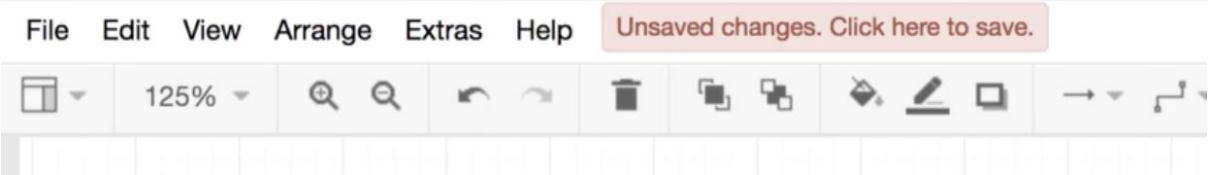
If a process is likely to take nine or more seconds, a progress bar may be needed, like this:

If a process is likely to take about a minute or longer, give a time estimation, like shown here, because this will enable the user to manage her time better by for example start to do something else while the process goes on, and have a rough idea when she should return:

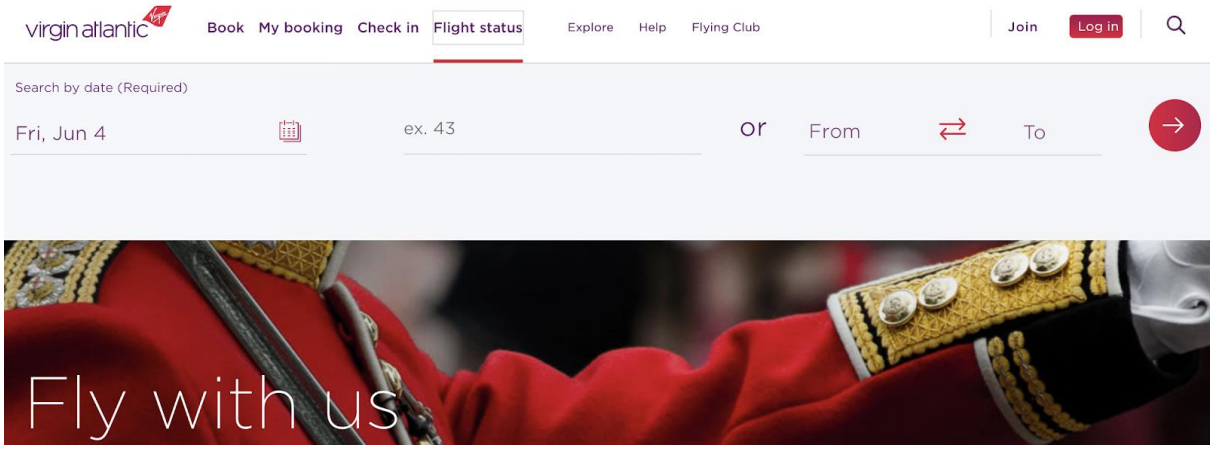


The status indication may also be whether the process is going on or standing still, switched on or off, if something is done or yet outstanding.

Here you can see that the spreadsheet software warns the user that the latest changes are not saved and still must be saved



Status indication is also used to indicate where you navigated to or on what page you are, for example, if you are now on the home screen or on the about page. For example, on this screen it is clear that we are viewing the flight status page because on the top menu bar, the “Flight status” menu item is clearly marked with a frame and a red underline.



The status indication may also be on what step in a process you are at present and whether you successfully completed the previous steps.

Indicating the status of their request or the process is important because when users know the current system status, they learn the outcome of their prior interactions and can predict the next steps. Predictable interactions, as well as open and continuous communication, let the user trust you.

2. Match between system and the real world

Use words, phrases, and concepts that are familiar to the user, based on how people perceive the natural world. Avoid jargon or terms used within your organisation only, rather use descriptive communication that the user will understand from their point of view. It is also a good idea to follow conventions, making information appear in a natural and logical order.

There are two important ways to match a system with the real world, namely natural mapping and skeuomorphism.

Natural mapping is the way in which people use or interpret space to represent actions, for example, to increase the volume or amount of an item it would make sense to indicate this with an up arrow or to drag a slider upwards, while it makes sense to decrease it with a down arrow button or downward slide on a slider.

In Western cultures that read from left to right and top to bottom, it is more effective to position back buttons or arrows to the left and forward or next buttons or arrows to the right.

A very practical and real example would be a map of a mall that is a representation of the actual space but from the user’s perspective. The physical shops to the left in the mall are positioned on the left on the map too. Shops to the actual back at the bottom of the map and shops to the front on the map’s top.

Another classic example is a four-plate stove where the controls for each plate is arranged in the same way that the actual plates they represent are.

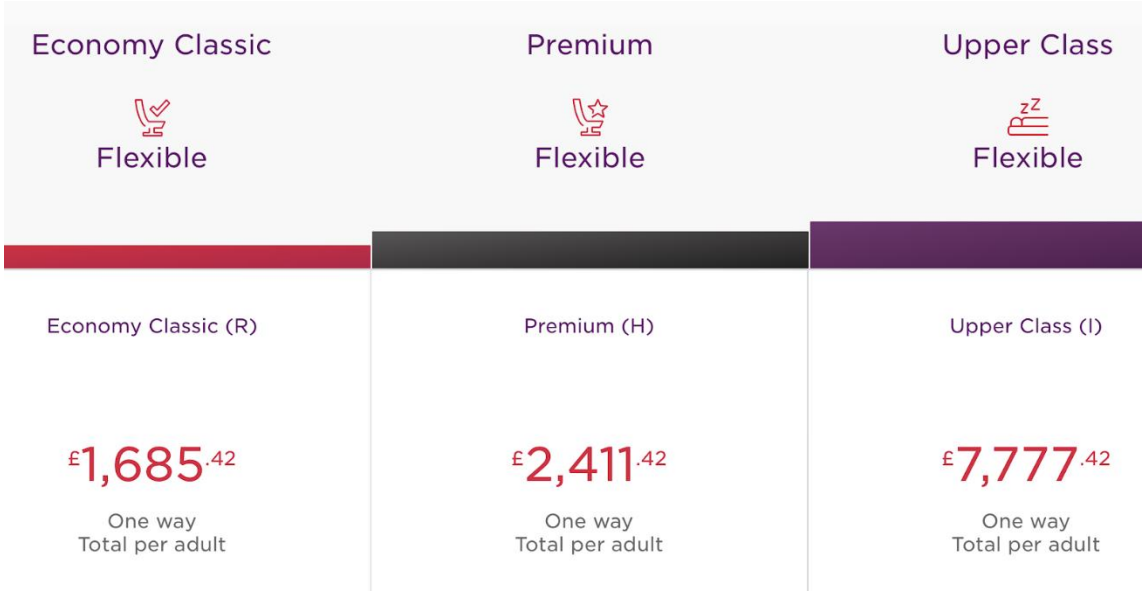
The other way to match the system to the real world is through skeuomorphism.

Skeuomorphism is a design that looks very much like the real thing it represents.

Skeuomorphism is very common in icon design. For example, an editing button that looks like a pen, an eraser tool that looks like a rubber eraser or an account icon that looks like a credit card.

Skeuomorphism is especially useful for users who are not digitally savvy, or for new functionalities that have so far been part of the physical rather than the digital world.

Here you can see examples of both natural mapping and skeuomorphism on the Virgin Atlantic site to indicate the different flight classes. The economy class icon shows a seat icon that looks like a real seat, with a tick mark most probably indicating it's OK or sufficient to pass. Under premium, we have a star next to the seat indicating a superior seat or VIP option. Under the upper class, you will find an icon of a sleeping person. We know this since we have been trained to identify the 'zzz' icon as sleeping. The upper class then most probably indicates the luxury of space to sleep in as well.



Regarding this heuristic of finding a match between the system and the real world, it is important to note that the frame of reference differs widely for different people depending on their culture, work or industry, age, or institutions they are part of. For example, metaphors in language, icons and even the possible meanings of colour are very different between languages and countries. To gain an understanding of how you can efficiently match the system to the real world as perceived by your users, you must conduct UX research, which will be covered in a later lesson.

It is important to take this heuristic very seriously because it can dramatically increase the usability of your digital product. When the user controls represent real situations it's easier for users to learn and remember how the interface works. This allows the interface and experience to feel intuitive to the user. It enables users to understand the meaning without having to go look up or memorise what it means.

3. User control and freedom

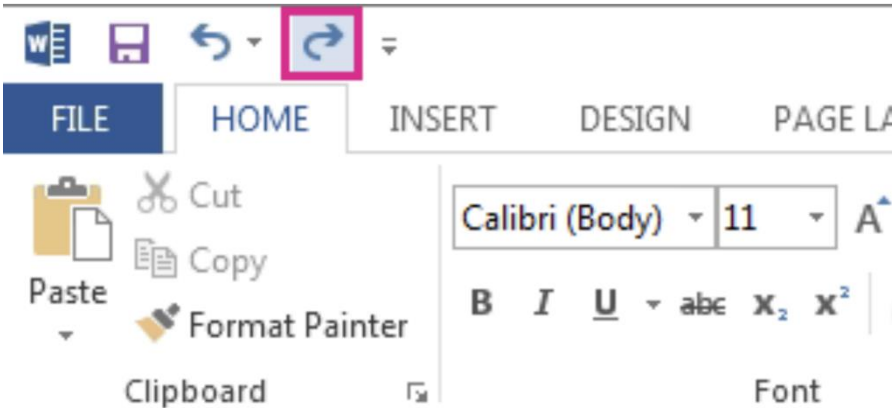
When customers visit a supermarket, they want to freely explore the aisles and the products on offer without being forced into a specific direction. They may go down the cosmetics aisle to search for toothpaste, realise it's not there and discover it in the health section. Along the way, they may have found another product they are interested in. Users want the freedom to explore and peek into something and retract the moment they lose interest for whatever reason. Then they want to get quickly back to where they originally were and continue exploring further. In short, customers want the freedom to explore, to try things without commitment or to make mistakes but then be able to quickly get back on track again.

The same applies to users of a digital product. They want the ability to navigate through the menu but if they feel lost, they want to get quickly to where they feel comfortable. They want to see what something is about without signing in or buying it and want to be able to drop it immediately if it is not the information or product that they want. As users often perform actions by mistake, they want the ability to recover from these mistakes instantly and easily. They need a marked "emergency exit" to leave the unwanted action without having to go through a long, tedious process.

When it's easy for users to back out of a process or undo an action, it creates a perception of freedom and confidence. Exit options allow users to remain in control of the system and avoid getting stuck or feel they are funnelled into the wrong direction.

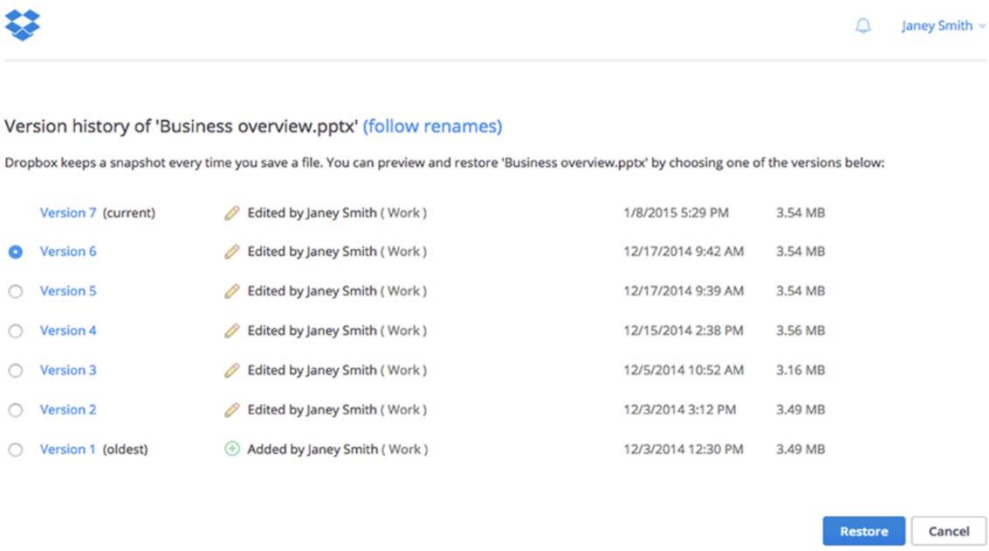
In its simplest form, a back and forward button must be available so that the user can easily move back or forwards through a process to feel in control. This must be accompanied by some form of mapping or status indication of where the user is in the process, like a progress stepper, page number or bread crumb indicator. This overlaps with the first heuristic discussed earlier namely visibility of system status.

Another very efficient way to give users control is to have an undo and redo button where appropriate. Preferably the system should allow a user to go back or forth several steps, not just a single step. Here you can see Microsoft Word's famous undo and redo buttons on the Quick Access toolbar. Note that the Redo button only appears, as highlighted in this screenshot, after you've undone an action. It is good practice to hide or let a button appear disabled if it is not relevant in context, like the cut and copy buttons in this case. This reduces the mental overload of the user, which relates to the fifth UX heuristic namely error prevention which will be discussed shortly.



For some digital products undo and redo buttons are not enough to facilitate user control and freedom. Some creations can take several hours, days, months or even years to complete or to improve over time. For example, a document, a spreadsheet, a piece of software or a work of digital art. For these systems, there should be a version control system where a user can revert to an earlier version or jump to a later version listing the different versions along a timeline with a date and time stamp. It is good practice to avoid a date only but rather provide a time because users often make major changes on the same day but at different times of the day. If possible, highlight or provide an auto-summary of the differences between the versions.

Here you can see version control options where a user can select a version, they may want to revert to by selecting the appropriate option button on the left, then click on the Done button. Also note the cancel button, which is important for the principle of user control and freedom, which will be discussed now.



Apart from back and forward navigation, undo, redo or version control mechanism, there should also be a way to cancel or abort an action or process, like a cancel button. Similarly, there should be ways to completely sign out of the app, switch off or shut down the appliance, and in line with the principle of visibility of system status, the user must see that he successfully exited the system. Otherwise, the fear of unintended continuation of a session may waste battery life in case of an appliance, allow unauthorised hackers to make transactions on a banking app or even allow others to read your mail.

All these possible abilities related to user control and freedom must be clearly marked or visible so that the user can see that it's available.

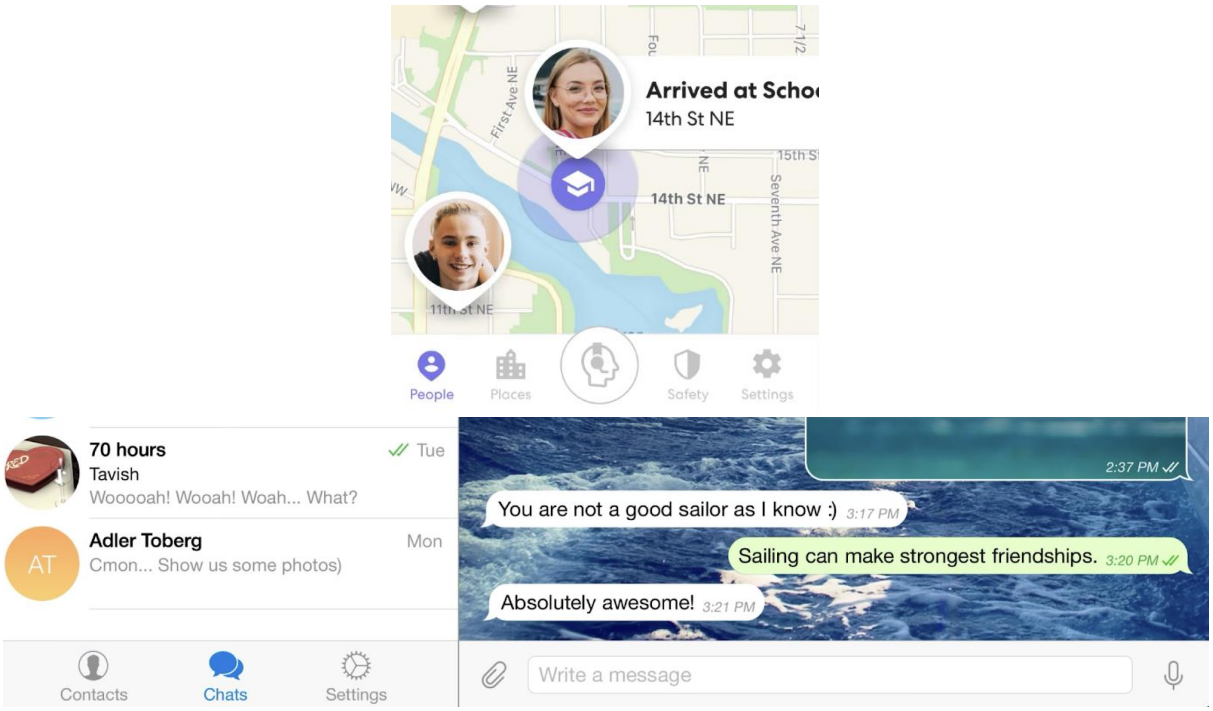
4. Consistency and standards

The design of user interfaces must be consistent across screens within the same app, product, or brand. This is referred to as internal consistency. If a user can swipe images on one screen, they expect to be able to swipe images on all screens with similar dimensions or spacing. If the forward button occurs at the bottom right through one process, users will search there for the forward button in all the processes in the same app.

Consistency is not only important within the same app or brand, but also between or across different apps of different brands or organisations. This is referred to as external consistency. It is important to follow commonly accepted UX design conventions used by other popular apps your users regularly use.

In a future lesson, we will learn about the 'laws of UX', of which Jakob's Law is one. This law states that people spend most of their time using digital products other than yours. Users' experiences with those other products set their expectations. Users learn how to use interfaces and interaction patterns from large social media platforms, well-known software, popular games and major e-commerce sites.

In these screens of the Life360 location app and the Telegram messaging app, like so many of the more popular apps, the people or contacts and settings icons looks very similar and familiar. You have probably seen the other icons like the shield for safety or security, a building for places, speech bubbles for chat, paper clip for attachments, and microphone to speak, on so many other apps that you instantly know what those means. This represents good external consistency in design and make an interface much easier to use.



If you don't maintain internal or external consistency your users will have to learn your way of doing things, which takes time and might be forgotten. Maybe your user will not even discover how your way works and will never be able to learn it in the first place. This forces your user to experiment, think and remember, which is not good at all because you want your interfaces to be as easy and intuitively to use as possible.

5. Error prevention

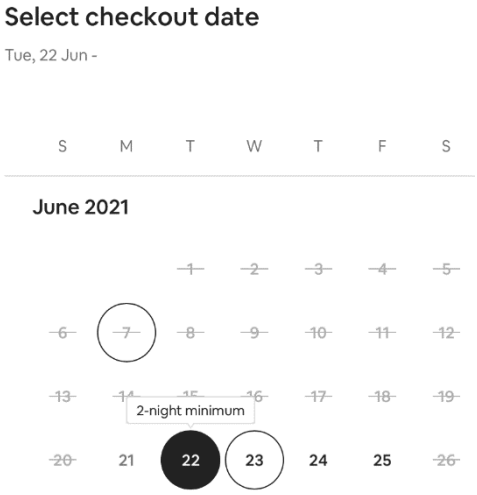
There are two types of errors that a user can make: slips and mistakes. Slips are accidental errors made because the user did not focus full attention while performing the action and would have known it was an error if proper attention was paid.

Mistakes are conscious errors made because the user did not know how to perform the task accurately. Maybe the user had no idea how to do it or had a misconception of how it should be done.

As discussed previously, helping users to recover from errors including helpful error messages are important. However, it is way more effective to assist the user in avoiding these errors in the first place.

One way to help prevent users from making mistakes is to eliminate error-prone conditions; make it difficult or impossible to make mistakes. This can be done by providing helpful constraints, accepting only likely correct options. For example, if a user must input their age, only accept real numbers, and only between a reasonable age range like between eighteen and hundred and thirty.

Here you can see how Airbnb guides the user to easily provide correct input when selecting check-in and -out dates for a place to stay over. Unavailable dates have a strike-through and you can't select a check-out date on the same date or earlier than the check-in date. If you select a too short period to stay, you will get a message notifying you what the minimum stay period for the place is.



Another way to make it difficult to make mistakes is to provide well-estimated default values. For example, if your user has to specify a delivery address, make the last or usual delivery address the default value, which the user can change if needed.

Yet another way to make it difficult to make mistakes is to provide a set of most likely options. For example, if the user has to input a delivery address, provide a shortlist of likely options to choose from, along with another option in case your guesses are not correct. Good options to guess may include the current location, work address and home address on record, and previously selected address.

Another way to help prevent users from making errors in the first place is to check for likely accuracy and present users with a confirmation message before they commit to the action. A well-known example is if you write an email and refer to an attachment, yet you did not attach a file by the time you hit the send button, Gmail will politely remind you to add an attachment. Gmail will also check the subject field of the email, and if you did not write a subject heading, it will ask you if you are sure you want to send the email without a subject heading. Confirmation messages are especially important for potentially costly or major mistakes. If you hit the discard button after you worked for an hour on filling in a form, or if you want to transfer a large sum of money to a recipient you never sent such a large amount to in the past.

Yet another way to help prevent users from making errors in the first place is to prevent cognitive overload, or not to overwhelm the user with too much information, too difficult or mentally exhausting tasks, and to do much of the thinking and calculation on their behalf.

Also, where appropriate, give helpful advice such as better priced options before a user buys an expensive item, or present a checklist of things to consider before a user choose an investment product.

It may also be handy to gather relevant information in one place before a user decides. A calendar app may show public holidays and forecasted weather information before a user plans for outdoor activities or meetings. This may help a user not to request a meeting on a public holiday or arrange a picnic on a rainy day.

Lastly, the humble undo button, as discussed earlier, is also important to help a user not to make an error, or rather not to follow through with it.

Did you know?

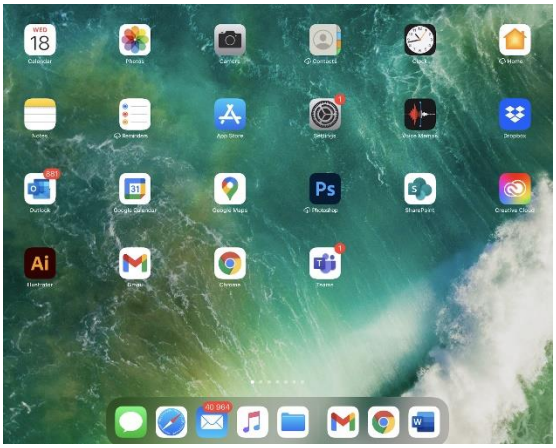
Business insider made some interesting discoveries based on banner ads. You know those terrible banners that flash or are just simply in your face? Well, people in general disregard and ignore these banner ads to such an extent that you are more likely to climb Mount Everest or even win the lottery!

6. Recognition rather than recall

Do not expect users to remember things. Rather provide them with clues making it easy to recall information. Remembering takes more time and effort, makes a system much more difficult to understand, and some people may not be able to remember well, therefore accessibility and inclusiveness may become a problem.

It is much easier to recognise information, concepts, or objects than it is to recall them. For example, an open-ended question in an exam asking what the capital city of Argentina is, is more difficult than a multiple quiz question providing you with a few options to choose from. Therefore, modern computer operating systems like Android, Windows and Mac with menu bars and icons are much easier to use than old computer command-line interfaces where the user are confronted with only a flickering text cursor on an empty screen. You were forced to know exactly what to type and how to spell it 100% correctly otherwise nothing will happen except maybe some unhelpful error message.

Here you can see the modern iOS operating system interface, known to be very easy to use to get the most out of the powerful iPad operating system

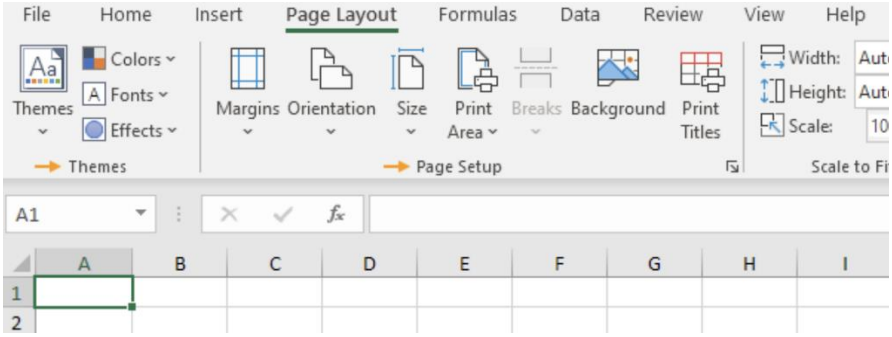


Compare it to the following command-line interface as an alternative way to communicate with the operating system, which required so much knowledge and memorised commands that only experienced programmers can use it productively.

The more memory hooks or queues are provided, the easier it is to recall. For example, an open-ended question or task that provides only one memory hook is more difficult than the option to choose from multiple hooks which greatly enhances recall.

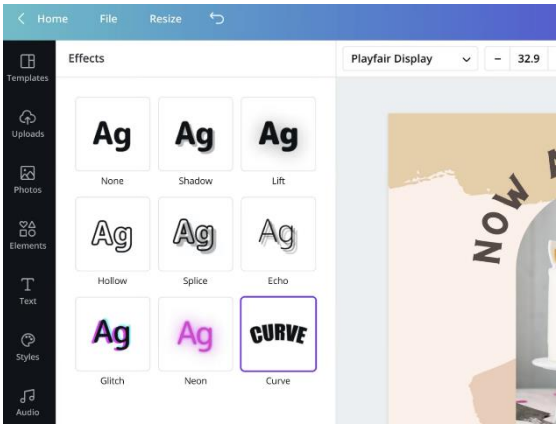
To provide three or more memory hooks is even better. An excellent example of helping users recognise functionalities, menu items or a way to navigate through and use a system, is to use a combination of descriptive label text in combination with an intuitively understood icon. Both the label or words, and the icon or simple illustration, help the user to easily recall how to interact with the system.

The following example shows Microsoft Excel's ribbon or menu bar where short, descriptive labels and meaningful icons are combined to make recognition very easy.



Compare it to the Adobe Illustrator toolbar which is much less intuitive to use because the differences between similar icons, and the icons in general, are not intuitively understood, and there are no labels, making it very difficult for new users to learn

So, due to poor UX, competition for Adobe Illustrator stepped in with a much more intuitive toolbar. See how well Canva uses the recognition rather than recall principle compared to Illustrator's interface. This enabled new users to be productive much faster.



Another way to create an additional memory hook is by providing more than one way to navigate to or accomplish a task. For example, in a shoe e-commerce store, hiking boots may be listed under both the sports shoes section as well as under the leisure shoe section, because hiking may be considered by some as a type of sport, while others will consider it as a leisure activity.

Overall, reduce users' reliance on memory by making elements, actions, and options visible, not only for a particular step but across the entire journey. The user should not have to remember information from one part of the interface or step to another. Key to being memorable is to be able to visually see not only the current options but also the larger context. For example, the user should see where they are in the system through what we like to call breadcrumbs or an active menu indication, like 'I'm now in the female hiking boots section, which is part of the general hiking boots section. The user should also be able to easily remember information from previous or earlier steps by providing a stepper indicator where the user can easily navigate back and forth without the need to make notes to remember information they entered for example.

The system should also enable the user to store information that may be required later, like providing the ability to add notes, or storing documents containing information that may be needed later. Information must be easily retrieved when needed.

Another example is a banking app that allows a user to save beneficiaries, transaction history and to make similar or repeat payments. This also points to yet another way to increase the ability to recall within the greater context, namely, to show the user tasks they performed earlier or list products they viewed in the recent past.

Another very powerful way to reduce reliance on memory is to guide a user on how to operate your system in the form of helpful tips, information buttons and other forms of assistance to use within the step or context, when and where needed. This is opposed to expecting a user to remember steps or learning how to use them prior.

A tricky but very common and important issue that requires users to remember something, is passwords. If possible, use modern but trusted technologies to reduce or eliminate the use of passwords, for example, biometrics such as fingerprint or facial recognition.

Lastly, if a user must recall, minimise the information that a user has to remember, and assist them if they can't recall something. Good examples may be a powerful search or filter capability to help a user finding something they can only vaguely describe, or a 'Forgot my password' button.

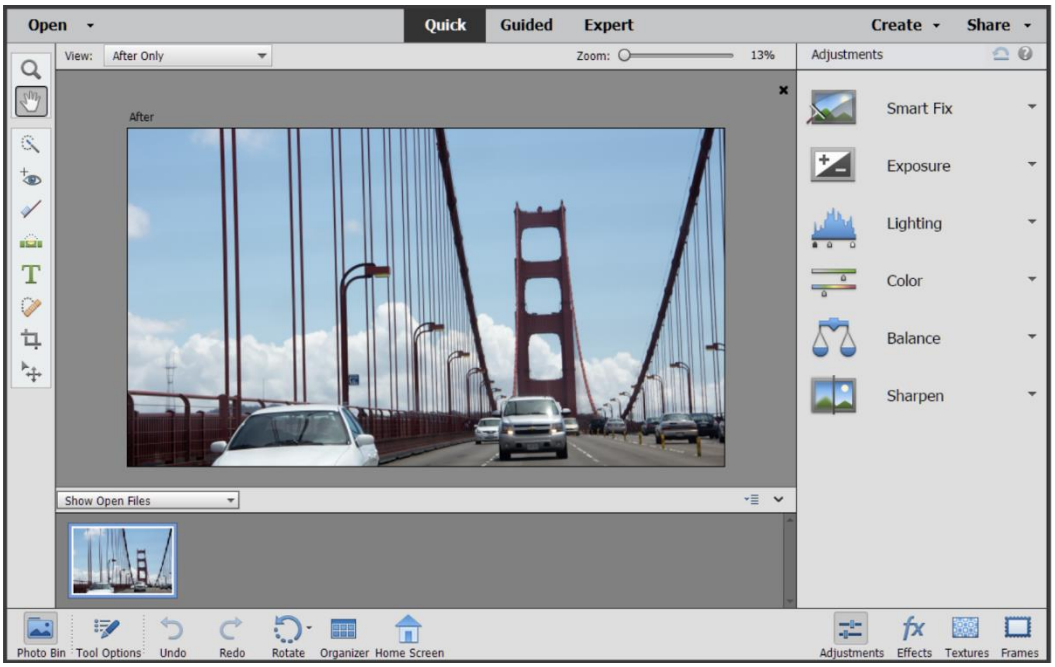
7. Flexibility and efficiency of use

Different users have different ways of doing things. This is because of personal preferences, how people learnt originally, due to differences in user's mental models or the way they think of or understand something. It is important to research such differences amongst your target users and provide alternative ways to do it. For example, in a banking app when a user wants to pay someone, some users prefer to go to the payments section, others want to click directly on a person to pay, yet others prefer to first select an account from which the payment must be made. It is a great idea to enable users to do it in any of these ways.

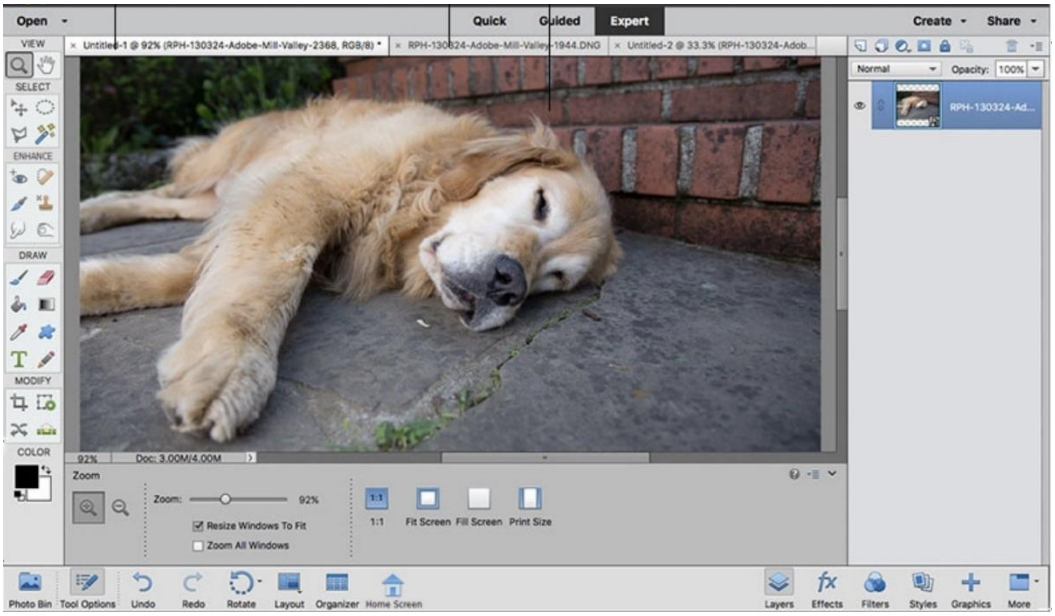
Another important consideration is the level of skills or experience a user has with digital technologies. New or very basic users generally prefer a very simple interface, while advanced users want advanced features, and can handle more complex interfaces. The app must be able to cater for your target users on both ends of the experience and skills spectrum. One way to accomplish this is through personalisation of the interface, where the interface is adapted according to the skills level and need for advanced functionality for a particular user.

Another way to accommodate the skills level and level of complexity that must be handled is through customisation. Enable users to customise or make changes to the interface and the way they do things, to meet their particular needs and preferences. A good example is to enable users to re-arrange the position or order of apps or quick links on the home screen.

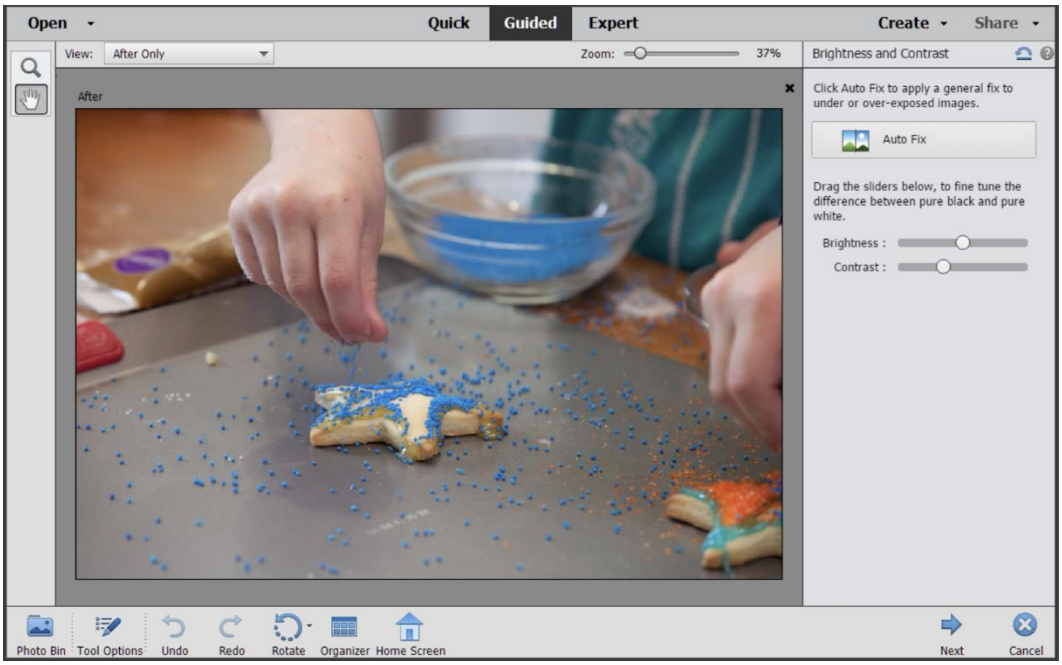
Adobe Photoshop enables users to switch between basic and advanced interfaces. First, look at the 'Quick' or basic mode which has a simple interface. Note that the mode can be set on the top menu bar.



Here is the ‘expert’ or advanced mode, which caters for the needs of power users:



The ‘guided’ mode is also an example of the implementation of the flexibility of use principle. Users who prefer to be guided through the photo editing process enjoy this mode,



Another way to accommodate advanced or power users is with the help of accelerators like shortcuts. Accelerators are features that make power users more productive. It helps them to perform their tasks faster and easier, especially tasks that they do frequently.

Examples of accelerators include the ability to bookmark, tag or classify content, products, actions, or features.

Accelerators may also include less obvious interaction mechanisms like gestures, such as swiping or shaking, that are quick and easy to perform but also well known to experienced users. Accelerators can also include keyboard shortcuts that a user can memorise, for example, pressing the control and c keys to copy.

A powerful accelerator can be to tailor or personalise the user interface, especially a home screen or dashboard on which a user spends a lot of time. More frequently performed tasks should be easier accessible.

For example, the information architecture (the way content is organised) can be according to logical grouping for new users and can be reshuffled according to the frequency of use for experienced users. Such personalisation can be manual, where the user can re-arrange content or items themself. But as machine learning progress and guessing what users will want to do or access next, the personalisation of the user interface can become more automated saving the user a lot of effort and knowledge to personalise the interface.

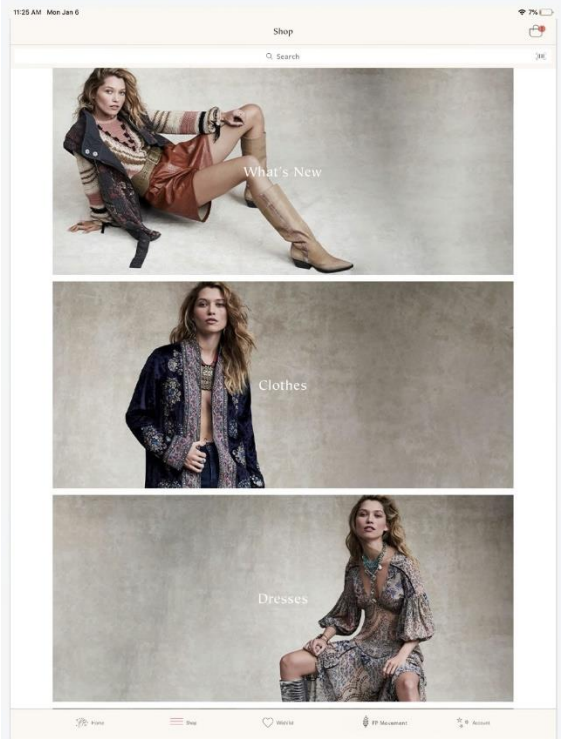
Related to automated personalisation of the user interface is a recommender system. A recommender system is a practice of highlighting the most useful options for the user. For example, an online shop may have a carousel component or list of recommended products that feature products which the user is more likely to be interested in, while the thousands of possibly irrelevant products are not displayed. This is to simplify the user interface and limit options, which are both important in user experience design. Such recommendations are performed automatically and could be based on very basic factors such as the user's age or other demographic criteria but should much more preferably be based on truly personalised data and machine learning algorithms that take much more and relevant behavioural data into consideration.

It is important to protect inexperienced users or users who prefer not to use any of these accelerators, from unintended actions. For example, an inexperienced user may easily delete an item unintentionally while swiping. Therefore, make it obvious how to 'undo' an unintended task triggered by an accelerator. Also, some people just don't want to use certain accelerators, like many people working every day on work processing software just don't like or won't remember keyboard shortcuts. Therefore, don't force a user to use accelerators.

8. Aesthetic and minimalist design

Having a simple, clean, or minimalistic user interface makes it easier for users to navigate and operate. It is therefore essential to show only relevant elements while hiding irrelevant elements. The less options there are, the easier it is to make a choice. This is because the more options or content there are, the more it competes for the user's attention, and the more time and effort it will take the user to find what they are looking for. This is referred to as noise and it increases the probability that the user will not find what they are looking for. This may even lead to the user abandoning the search. Information and choice overload is a common reason for abandoning a site, app, online purchase, or process.

Here is an example of a clothing e-commerce app embracing simplistic design, giving it an elegant and luxury look and feel while directing the user quickly towards appropriate products.

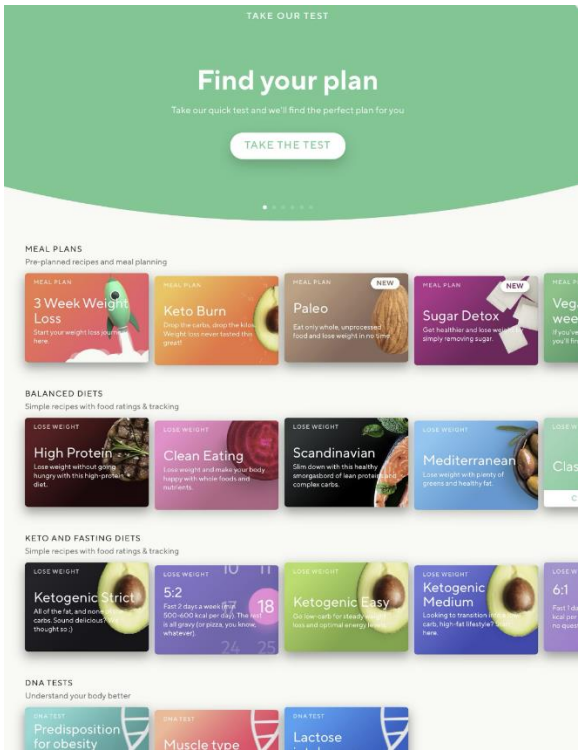


However, less relevant information must still be easy to find. Therefore, information architecture is so important. It allows users to quickly navigate through a page effectively. This is also why personalisation is important as it highlights the relevant content to a user's specific requirements.

The opposite of minimalistic design is a cluttered interface with an overload of irrelevant information, where the user has to dig deep to find what they are looking for. There are too many headings or calls to action that competes for attention, or too much text without being grouped under a few descriptive headings. This should be avoided. The rule of thumb here is to ask yourself: what can I remove to make my design more effective, rather than asking what I should add.

Sometimes it is difficult to eliminate or hide information, where it may make sense to provide a content-rich interface. One example is an interface for experienced or advanced users, or where it is very difficult to predict what the user would like to do amongst many possible options or content. Apart from having a good navigation structure or information architecture, such a large quantity of content can be dealt with by making clever use of visual hierarchy and grouping. Placing information in chunks that logically belongs to each other, with clear, short and descriptive headings that visually stands out, and letting the sections be ordered in a logical order that can be quickly scanned over to find relevant content, may also be very helpful.

This diet app, Lifesum, combines information richness with minimalistic design through clever, logical use of information grouping in card which is useful to both new and advanced users.



Do not haphazardly eliminate options and content. An over-simplified interface where a user may not easily see where to go to perform the desired task or get the information needed at the time may be very frustrating. Instead, prioritise content according to the task or need of the target user, preferably personalised at an individual level.

Did you know?

You probably would have picked this up by now, but UI design is the process of taking extensive, elaborative or complicated information and making it easy for others to grasp.

UI is all about problem-solving and this skill can be applied to everyday life too.

9. Help users recognise, diagnose, and recover from errors

Whenever something goes wrong, five things must happen:

- The user must first be made aware that not all is well
- Secondly, determine what the problem is
- Thirdly, solve the problem.
- The issue should be followed up to make certain the issue is fixed
- Finally, the system must be improved to help users to prevent the error from occurring again

Consider this scenario: a software developer will see a red squiggly line under a piece of problematic code. The code editor, VS Code, in this case, gives a very helpful suggestion to fix the issue, from which the user can also infer the cause of the problem. The user simply has to click on the suggested solution to have it be fixed automatically. It goes even further to offer to fix all other related issues as well.

```
export class ModeServiceImpl implements IModeService {
    public _serviceBrand: any;

    private readonly _instantiatedModes: { [modeId: string]: IMode; };
    private readonly _registry: LanguagesRegistry;
    Import 'LanguagesRegistry' from module "vs/editor/common/services/languagesRegistry"
    Add all missing imports
    private readonly _onDidCreateMode: Emitter<IMode> = new Emitter<IMode>();
    public readonly onDidCreateMode: Event<IMode> = this._onDidCreateMode.event;
```

Let's look at the first error handling component, namely being made aware that something did go wrong. This is usually in the form of a red colour highlighting, encircling or pointing to the problem area. Because some colour-blind people can't see red, and to make the message stronger, also add an icon representing an error, like a cross, exclamation mark, unhappy face or something more creative but still clearly understood.

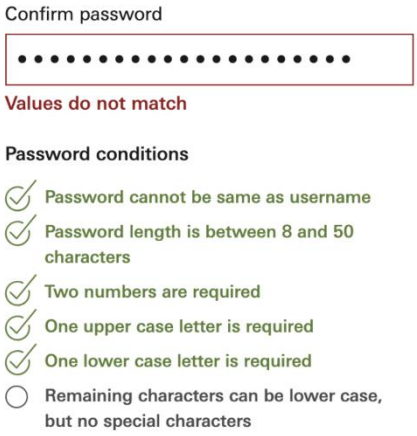
Regarding the second error handling component, namely, to tell the user what went wrong, giving clarity is important. The user must know exactly what went wrong and why it is an issue. Use plain, easily understood language, not technical jargon. Do not provide error codes because it may confuse users and is meaningless and frustrating if it appears without further helpful guidance. There may be exceptions when an error code may be helpful like for a technical audience or if it will be useful as a reference or description when the problem needs to be escalated.

Thirdly, the user needs to be provided with a solution to the problem. Avoid generic, non-helpful messages like 'contact your administrator'. Rather auto-generate a message to the specific person who can help with details of the problem auto-filled. In general, try to make the solution as simple as selecting or pressing a button so that the problem can be solved immediately with little effort.

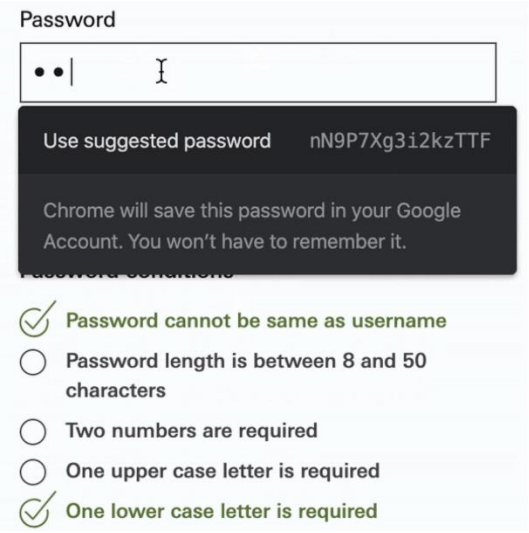
Next, follow up on whether the issue has been solved. For example, while the error case message was sent automatically to the administrator, a task flow or ticket may be auto generated that will allow tracking of the issue by the administrator's manager, possibly by the user, and closing of the ticket after being resolved.

Finally, log the problem so that it can be investigated. If an error carries a high risk like loss of life, or if it occurs quite frequently, it is essential to find the root cause and adapt the system to prevent the error from occurring again.

The following example from zuko.io can be analysed through the principle of error recognition and recovery as follows: the user can clearly see that the password values do not match. Helpful tips are provided in the form of a checklist of password requirements, which updates as the user types.



It goes further by suggesting a password that complies with all requirements which can be accepted with a single click. It even offers to store it safely so that the user doesn't need to remember it.



10. Help and documentation

Preferably, an app, software program or device must be so intuitive to use that no instructions or studying are required. Ideally, the system won't need any additional explanation beyond the clues build into the interface.

However, for very complex tasks additional help and instructions may be needed. Whenever a user may not know what to do, provide help when and where it's needed. Help can be in the form of live chat, a number to call, a tooltip, a frequently asked questions section, how-to articles, in-context help messages, information buttons, informative labels or instructions, links for more information, helpful error messages, or other means to get to a solution. Provide this help in the user's journey, right at the step where it will be required.

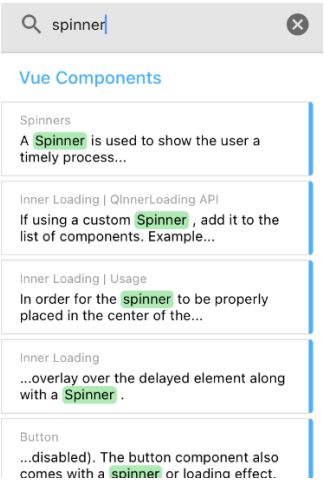
Another excellent way to provide help is in the form of examples and case studies, clearly illustrating how to use your product or perform your task in a particular context.

Well written, concise, and easy-to-understand documentation or tutorial-style instructions are very important for more complex software or systems. Make certain it's well written, supported with examples, illustrations, animations, videos and ready-made, adaptable solutions ready to be copied and pasted, like code snippets. Although well-written, simple text-based documentation may be very useful and appropriate, illustrations, graphics and most importantly, videos may be most effective. Even experienced developers need help. For example, code snippets and developer sandbox environments where they can experiment with code. Help and documentation content should be easy to search, focused on the user's task, concise, and list specific and clear steps that need to be carried out.

This example of formal documentation of the cloud computing giant AWS illustrates that it is possible to make very complex tasks doable with well-written documentation supported with illustrations and code examples.

Have a search field with intelligent search functionality so that the user can type a question and easily find the right answer. Although building your search functionality may sometimes be required, it is often best to simply make the instructions or help documentation available to search engines and integrate the search engine's search service to your site, hence, saving you a lot of unnecessary time and effort.

The following screenshot from the Quasar Framework cross-platform developer documentation illustrates how easy a search box can make it to find appropriate content in an extensive body of documentation:

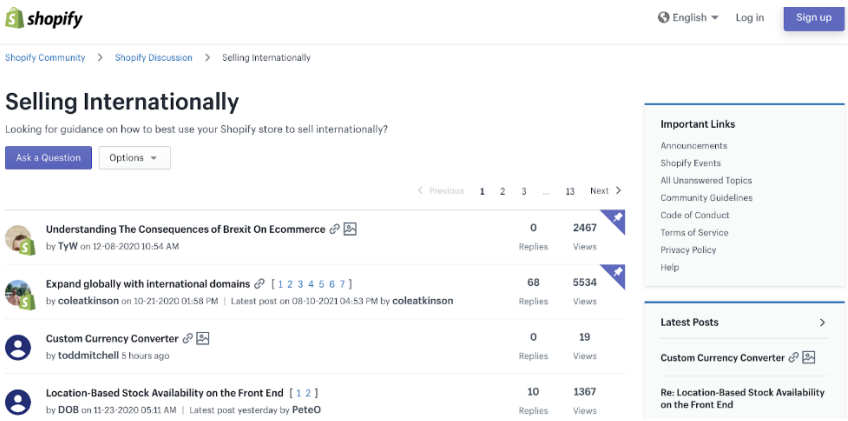


Another way to provide help is by writing a Questions and Answers (Q&A) section which addresses important and common user questions. Like in the case of most other forms of documentation, make it available to search engine crawlers so that it can be easily discovered on the internet, which will also increase marketing exposure through improved Search Engine Optimisation (unless you want your product to be hidden from the public).

Help should also be available through direct contact to a human or virtual assistant via instant chat, or a human assistant through email and a phone number. Ideally, provide the user with all these options because people differ in their preferred mode of requesting assistance.

Alternative help can be provided in the form of an online community centred around your product. This is viable in case you have a lot of users and/or a type of product that users feel passionate about. The users can help each other through community forums and user-generated content, such as how-to videos uploaded to the internet. These communities may be on your site, or you may provide answers to questions asked about your product in communities that do not belong to you.

This is an example of an active, vibrant community where e-commerce software-as-a-service Shopify's customers advise, support and encourage each other. Note that community interactions are classified by user's important issues and it orders issues by recency and helpfulness or popularity. Community support is an important contributing factor for Shopify's ongoing success.



If extensive training is required, in most cases it's best to provide it in the form of free online videos, not expensive in-person training. Most people prefer easy-to-see screencast videos. Usually, it's best to make money from using the software, not from selling courses to use the software, therefore make it free so that more people can learn to use your software.

Good onboarding is an excellent way to support help and documentation, which will be addressed in another module.

Lesson challenge

The lesson challenge I would like to leave with you today is to find a list of shortcuts for your favourite application. This can be Microsoft Word, PowerPoint or even Adobe applications. Simply search the term xxx hotkeys and download it.

Now, take the hotkey sheet and memorise at least one shortcut that you think will make your task for this application quicker and more effective. Apply it and see how it works for you.

If you want to, make a habit out of learning and applying keyboard shortcuts. You will find that it really does make your tasks quicker.

Conclusion

In this module, we've learned about the 10 principles of Jakob Nielsen's framework for doing a basic UX audit or evaluation. The Jakob Nielsen framework is not the only framework for conducting a UX evaluation. In a later lesson, we will learn about another checklist approach for evaluating or auditing user interfaces, namely Jon Yablonski's Laws of UX, which will fast-forward your knowledge of visual design as well.

Reference

Nielsen, J. 2020. 10 Usability Heuristics for User Interface Design. Accessed from: https://www.nngroup.com/articles/ten-usability-heuristics/

Nielsen, J. 1994. How to Conduct a Heuristic Evaluation 2021. Accessed from: https://www.nngroup.com/articles/how-to-conduct-a-heuristic-evaluation/ >.

Nielsen, J., and Landauer, T. K. 1993. A mathematical model of the finding of usability problems. Proceedings ACM/IFIP INTERCHI'93 Conference (Amsterdam, The Netherlands, April 24-29), 206-213.